ML17 Mistakes, overfitting and fair comparisons

Week 6, Trees and fair comparisons | Session 17 of 24 | Machine Learning | New library: sklearn, matplotlib

Today's question. When is a very high score a warning sign?

Objective. By the end: you can read a confusion matrix, spot an overfitted tree from its training and test scores, and explain why the held-back set must stay untouched until the end.

> **WHERE THIS IS USED**
> A fraud alert that blocks a real payment upsets a customer, and one that misses fraud costs money. Teams count both kinds of mistake. When would a very high score make you suspicious?

Before this session: ML16 Predicting a category with a decision tree. If you cannot explain the last example, use its error card with the instructor first.

Words for today

Word	What it means	Think of it like this
overfitting	Learning the quirks of the training examples so well that new examples are predicted worse.	Learning one route by heart and getting lost on a new road.
confusion matrix	A table that counts actual categories against predicted categories.	A tally sheet of right and wrong guesses, kept in boxes.
actual and predicted	Rows show what really happened; columns show what the model said.	The real result against the guess.
validation records	Held-out rows used to choose between models.	A trial run used to decide which plan to follow.
held-back set	The last rows, kept apart and used once after every choice is made.	A sealed envelope you open at the end.
peeking	Looking at the held-back set while you choose settings, which spoils it.	Reading the last page of a mystery first.

Block A. Read a confusion matrix

A model guessed five yes or no answers. Count the matches.

Predict. The matrix will have two rows and two columns. Predict which cells hold the correct answers. Write your guess on paper first.

Your guess, written before you run it

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
actual = [0, 0, 1, 1, 1]
predicted = [0, 1, 1, 0, 1]
matrix = confusion_matrix(actual, predicted, labels=[0, 1])
print(matrix)
ConfusionMatrixDisplay(matrix, display_labels=["no", "yes"]).plot()
plt.show()
```

Run it, then compare. The cell prints [[1 1] [1 2]]. Rows are the actual answers and columns are the predicted answers. The top-left cell is an actual no predicted no, and the bottom-right is an actual yes predicted yes. The correct answers are on the diagonal: 1 plus 2 is 3 out of 5. ConfusionMatrixDisplay draws the table as a picture.

Investigate.

1. Which cell counts an actual yes that the model called no?
2. Why read the axis labels before the numbers?

Modify. Calculate the accuracy from the matrix.

Block B. The tree that remembered

Train trees that may ask more and more questions, and print their training and test scores.

Predict. Predict which tree scores 100% on the training flowers. Write your guess on paper first.

Your guess, written before you run it

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
    test_size=0.25, random_state=7)
for depth in [1, 2, 3, None]:
    model = DecisionTreeClassifier(max_depth=depth, random_state=7).fit(X_train, y_train)
    print(depth, round(model.score(X_train, y_train), 3), round(model.score(X_test,
        y_test), 3))
gap = model.score(X_train, y_train) - model.score(X_test, y_test)
```

Run it, then compare. Each line prints the depth, the training score and the test score. The tree with depth None may ask as many questions as it likes and scores 1.0 on the training flowers, because it memorised them. Its test score does not rise with it. That gap between training and test is the sign of overfitting. score gives the accuracy.

Investigate.

1. Why is 100% on training flowers a warning sign?
2. Can we use the test score to pick the best depth?

Modify. Print the gap between the last tree's training and test scores.

Make

Draw the 2 by 2 matrix for the toy answers and label both axes. Count the right and the wrong predictions. Explain why trying many settings against the held-back set weakens it.

Save your work as ML17. Keep the worked example and put your own attempt below it. Write two sentences: what you changed and how you checked it.

Error card

You see	It means	Fix
The two kinds of mistake sit in the wrong cells	The axes were read the wrong way round: rows are actual, columns are predicted.	Label both axes before reading any cell.
The matrix has actual and predicted swapped	The two lists were passed to confusion_matrix in the wrong order. It expects the actual values first and the predicted values second.	Write confusion_matrix(actual, predicted) and check that the row totals match the real class counts.

Exit ticket

A great training score with a weaker test score means overfitting, and a held-back set is only fair if no choice has used it.

AI use follows AI_USE_POLICY.pdf. In weeks 1 to 6 no AI-generated solutions are used for exercises, labs or build tasks. Practise for 20 to 30 minutes between classes by repeating the Make task with one value changed. If you are stuck for 30 minutes, save the error and bring it to class. A file saved on your laptop has not been shared until you use the route in START_HERE.pdf.